

StoryForge

Requirements-to-Backlog agentic workflow — project overview

Pradyumn Awasthi · September 2026 · github.com/pradyumn/storyforge

1. What it is

StoryForge is a tool for business analysts and product owners. You paste the raw output of a discovery session — meeting notes, an interview transcript, a rough brief — and a pipeline of five specialised AI agents turns it into the artefacts a delivery team actually needs: a structured brief, atomic requirements that each quote the sentence they came from, epics and user stories with Gherkin acceptance criteria, an INVEST review with an automatic revise loop, and a gap report listing the questions to take back to the stakeholders. One click publishes the backlog to Jira Cloud.

The design premise: *the hard part is not generating stories — it is producing stories a team would accept, and being able to point at the sentence each one came from.*

The problem it solves

A BA's first week on any initiative is the same: read the notes, work out what is actually being asked for, write it up as requirements, break it into stories, and go back with the questions the notes left open. It is 60–70% of the effort on a new engagement and it is where scope goes wrong — a requirement nobody said, a 'should' upgraded to a 'must', a goal with no story against it. Large language models are good at the drafting and bad at the discipline: they invent, they smooth over ambiguity, and they write stories that read well and cannot be tested. StoryForge is built around that gap.

2. What it does

The pipeline, in the order it runs:

Stage	Produces	Checked by
Intake	Title, business goals, stakeholders and their concerns, constraints, out-of-scope items, open questions	Schema validation
Requirements	Atomic 'The system shall...' statements with MoSCoW priority, type (functional / non-functional / data / integration / compliance) and a verbatim evidence quote	Schema + string-match traceability: the quote must exist in the notes, or the requirement is flagged
Stories	2–6 epics; stories as role / want / benefit with 2–4 Given-When-Then criteria and links to the requirements they cover	Schema + Gherkin validity + role, size and coverage rules
INVEST review	Scores 1–5 on each INVEST dimension, pass/fail, concrete fix instructions, duplicate and conflict pairs	A story passes only if the model passes it AND every score ≥ 3 AND the deterministic checks are clean
Revise loop	Only the failing stories are rewritten using the reviewer's feedback	Bounded to 2 rounds; every round recorded in the trace
Gap analysis	Uncovered goals, missing NFRs, conflicts,	Schema validation

ambiguities, missing stakeholders — each with a question to ask

Outputs

- **BRD.md** — a business requirements document: goals, stakeholders, constraints, requirements with evidence, open questions, gap analysis, run summary
- **backlog.md** — epics and stories with acceptance criteria, INVEST scores, and a traceability matrix (requirement ↔ stories ↔ evidence)
- **Jira Cloud** — epics and stories created through the REST API v3, with parent links and formatted descriptions; publishing is idempotent, so re-running the same analysis never duplicates issues. A dry-run mode shows the exact payloads without sending them
- **CSV / JSON** — Jira-importable CSV and the complete raw result for downstream tooling

Interfaces

- **Web UI** — single page: paste notes, pick a model and prompt version, then browse Brief / Requirements / Backlog / Gaps / Trace / Export tabs
- **REST API** — POST /api/analyze, /api/publish, /api/export; GET /api/health; interactive docs at /docs
- **CLI** — `python -m storyforge.cli notes.txt --out out/ [--publish [--dry-run]]`

3. Why five agents, and why the guardrails are code

A single 'turn these notes into stories' prompt works on a demo and fails on a real transcript. Decomposing lets each agent see only what it needs — the story writer never sees the raw notes, it works from the approved requirement list, which is what stops it inventing scope. Every requirement must quote its evidence, and that quote is checked by string matching, not by asking another model. A separate reviewer scores stories against INVEST and feeds only the failures back to the writer, for at most two rounds. Prompts are versioned (v1 → v4) and never edited in place, so every change can be measured against the same rubric.

4. Measured, not asserted

Quality is scored on a golden set of 8 synthetic discovery transcripts across 8 domains — 83 labelled requirements, 16 'distractor' sentences that sound like requirements but contain no ask, plus labelled exclusions and constraints. The rubric is deterministic and was fixed before any prompt was tuned. Results on Gemini 2.5 Flash with the v3 prompts (7 of 8 transcripts complete at the time of writing):

Metric	v3 result	What it means
Traceability (evidence quote found in notes)	99.4%	176 of 177 requirements quote the source verbatim — the model is not inventing
Requirement recall	82%	Share of labelled asks the model found
Story pass rate after the revise loop	88%	Stories the reviewer + deterministic

		checks accept as sprint-ready
Gherkin-clean stories	98%	Acceptance criteria with observable outcomes, no 'works correctly'
Generic 'as a user' stories	0%	Every story names a real role
Schema repairs needed	0	Every model answer validated against its contract first time
Requirement precision (strict rubric)	59%	Mostly over-splitting — one ask becoming two requirements — not hallucination; this is what prompt v4 targets
Distractor leakage	21%	3 of 14 anecdotes became requirements — also targeted by v4

The full method, every prompt iteration with before/after numbers, the provider decisions (including two failed attempts on Groq's free tier) and the failure analysis that motivated v4 are in eval/DECISIONS.md in the repository.

5. Tech stack

Layer	Choice
Language / runtime	Python 3.10+; type-hinted throughout
Data contracts	Pydantic v2 — every agent output validates against a schema; failures trigger one repair pass with the exact validation errors
LLM providers	Gemini 2.5 Flash (primary) and Groq gpt-oss-120b (fallback), called over plain HTTPS with httpx — no vendor SDKs; JSON mode; fallback router with rate-limit-aware retries and a client-side tokens-per-minute throttle
Offline backend	A deterministic stub so the whole pipeline, the tests and CI run with no keys and no network
API	FastAPI + Uvicorn; OpenAPI docs; daily budget on live runs and an admin key for live Jira publishing (public-demo guards)
Web UI	Single-file HTML/CSS/JavaScript, no framework; light/dark; six result tabs
Integrations	Jira Cloud REST API v3 (Atlassian Document Format descriptions, parent links, idempotent by label + summary); Markdown and CSV exporters
Evaluation	Custom harness: golden set, deterministic scorer, per-version JSON reports, resumable runs, README table generator
Quality tooling	pytest (43 offline tests), ruff, GitHub Actions CI, 14-step UAT script, preflight script for local and deployed instances
Packaging / deploy	Dockerfile; one-command deploy to Hugging Face Spaces (creates the Space, sets secrets, pushes, polls the build, runs remote preflight)
Docs	README, PRD for the product itself, architecture (with Mermaid diagram and a context-engineering table), UAT, decision log

Cost: zero paid APIs. That is a product constraint (cost per analysis must round to zero for a portfolio tool) and it forces the guardrails to be deterministic rather than 'ask a bigger model'.

6. How to talk about it

- **As business analysis:** it encodes BA discipline — traceability to source, MoSCoW, INVEST, Gherkin, coverage — as rules a machine has to satisfy, and then measures whether it did.
- **As AI solution design:** decomposition, per-agent context engineering, schema-constrained outputs, bounded agent loops, deterministic guardrails, versioned prompts, and an eval that decides which prompt ships.
- **As delivery:** a real integration (Jira), a real API, tests and CI, a UAT script, a hosted demo, and a decision log that records what went wrong before it went right.